Amazon EC2 Auto Scaling Group — Important Interview Questions & Answers

1. What is an Auto Scaling Group?
Answer:
An Auto Scaling Group, or ASG, is an AWS service feature used to automatically manage the number of EC2 instances running for an application.
It can launch new EC2 instances when more capacity is needed and remove instances when they are no longer needed. It also helps maintain the required number of healthy instances.
For example, suppose our application normally needs 2 EC2 instances. If traffic increases, the Auto Scaling Group can launch more instances. If traffic decreases, it can reduce the number of instances.
Example:

Auto Scaling Group
↓

↓ ↓ ↓
EC2-1 EC2-2 EC2-3

Simple ga gurthupettuko:
ASG = **Automatically launch + remove EC2 instances + maintain capacity**

---

2. Why do we use an Auto Scaling Group?
Answer:
We use an Auto Scaling Group to automatically maintain the required number of EC2 instances.
It helps improve application availability and allows the application to handle changes in traffic without manually launching or terminating instances.
For example, if one EC2 instance becomes unhealthy, the Auto Scaling Group can terminate the unhealthy instance and launch a replacement.
Example:
EC2-1 → Healthy
EC2-2 → Unhealthy ❌
↓
Auto Scaling Group
↓
New EC2 launched

Simple ga gurthupettuko:
ASG = **Keep the required healthy EC2 instances running**

---

3. What is the difference between EC2 and Auto Scaling Group?
Answer:
EC2 is the actual virtual server where our application runs.
An Auto Scaling Group manages a group of EC2 instances and can automatically launch or terminate instances based on our configuration.
For example:
EC2
↓
One virtual server

Auto Scaling Group
↓
Manages multiple EC2 instances

Simple ga gurthupettuko:
EC2 = **Server**
ASG = **Manages EC2 servers**

---

4. What are Minimum, Desired, and Maximum Capacity?
Answer:
These are three important settings in an Auto Scaling Group.
Minimum Capacity is the minimum number of EC2 instances the ASG should maintain.
Desired Capacity is the number of EC2 instances we normally want running.
Maximum Capacity is the highest number of EC2 instances the ASG can launch.
For example:
Minimum = 2
Desired = 3
Maximum = 5
Normally, the ASG tries to keep 3 instances running. It should not go below 2 or above 5 based on these capacity limits.
Example:
Min → 2
Desired → 3
Max → 5

EC2-1
EC2-2

EC2-3
Simple ga gurthupettuko:
Min = **Lowest**
Desired = **Normally required**
Max = **Highest**

---

5. What is Desired Capacity?
Answer:
Desired Capacity tells the Auto Scaling Group how many EC2 instances we want running normally.
The ASG tries to maintain this number unless scaling or other events change the capacity.
For example, if Desired Capacity is 3, the ASG tries to keep 3 healthy EC2 instances running.
Example:
Desired Capacity = 3

ASG
↓

EC2-1 EC2-2 EC2-3
Simple ga gurthupettuko:
Desired Capacity = **Normally required EC2 count**

---

6. What is Minimum Capacity?
Answer:
Minimum Capacity defines the minimum number of EC2 instances that the Auto Scaling Group should maintain.
The ASG should not normally scale below this configured minimum.
For example, if Minimum Capacity is 2, the ASG keeps at least 2 instances according to the scaling configuration.
Example:
Minimum Capacity = 2

ASG
↓
EC2-1 EC2-2
Simple ga gurthupettuko:
Minimum = **Lowest number of instances**

---

7. What is Maximum Capacity?
Answer:
Maximum Capacity defines the highest number of EC2 instances that the Auto Scaling Group can launch based on its scaling configuration.
It prevents the group from growing beyond the configured limit.
For example, if Maximum Capacity is 5, the ASG can scale out up to 5 instances.
Example:
Maximum Capacity = 5

EC2-1
EC2-2
EC2-3
EC2-4
EC2-5
Simple ga gurthupettuko:
Maximum = **Highest number of instances**

---

8. What is Scaling Out?
Answer:
Scaling out means increasing the number of EC2 instances when the application needs more capacity.
The Auto Scaling Group launches additional instances based on the scaling policy.
For example, if CPU usage becomes high because of increased traffic, the ASG can launch more EC2 instances.
Example:
Before:

EC2-1
EC2-2

↓ Traffic increases

After:

EC2-1
EC2-2
EC2-3
EC2-4

Simple ga gurthupettuko:
Scale Out = **Add more EC2 instances**

---

9. What is Scaling In?
Answer:
Scaling in means reducing the number of EC2 instances when the application does not need as much capacity.
The Auto Scaling Group can terminate extra instances according to the scaling policy and capacity settings.
For example, if traffic becomes low, the ASG can reduce the number of instances.
Example:
Before:

EC2-1
EC2-2
EC2-3
EC2-4

↓ Traffic decreases

After:

EC2-1
EC2-2
Simple ga gurthupettuko:
Scale In = **Remove extra EC2 instances**

---

10. What is an Auto Scaling Policy?
Answer:
An Auto Scaling Policy defines when and how the Auto Scaling Group should increase or decrease the number of EC2 instances.
The policy can use metrics such as CPU utilization or application load.
For example, we can configure a policy to increase capacity when average CPU usage stays above a certain level.
Example:
CPU Usage
↓
High
↓
Scaling Policy
↓
Launch EC2
Simple ga gurthupettuko:
Scaling Policy = **When and how to scale**

---

11. What is Target Tracking Scaling?
Answer:
Target Tracking Scaling tries to keep a selected metric around a target value.
The Auto Scaling Group automatically adds or removes instances to maintain that target.
For example, we can set average CPU utilization to 50%. If CPU goes higher for a sustained period, ASG can add instances. If it goes lower, ASG can remove instances when appropriate.
Example:
Target CPU = 50%

CPU increases
↓
Add EC2

CPU decreases
↓
Remove EC2
Simple ga gurthupettuko:
Target Tracking = **Keep metric near target**

---

12. What is Step Scaling?
Answer:
Step Scaling changes the number of instances based on how much a metric crosses certain thresholds.
Different levels of the metric can cause different scaling actions.
For example:
CPU > 60% → Add 1 instance
CPU > 80% → Add 2 instances
CPU > 90% → Add 3 instances
This allows scaling actions to become larger when the problem is more serious.
Simple ga gurthupettuko:
Step Scaling = **Different metric levels → Different scaling actions**

13. What is Scheduled Scaling?
Answer:
Scheduled Scaling changes the capacity of an Auto Scaling Group at a planned time.
It is useful when we already know that traffic will increase or decrease at a specific time.
For example, if an application receives high traffic every day from 9 AM to 6 PM, we can increase the desired capacity before 9 AM.
Example:
8:30 AM
↓
Increase capacity
↓
9:00 AM
↓
High traffic
Simple ga gurthupettuko:
Scheduled Scaling = **Scale at a planned time**

---

14. What is Predictive Scaling?
Answer:
Predictive Scaling uses historical traffic and usage patterns to predict future demand and adjust capacity in advance.
It is useful when application traffic follows regular patterns.
For example, if traffic regularly increases every morning, predictive scaling can prepare additional capacity before the expected traffic increase.
Example:
Past Traffic Data
↓
Prediction
↓
Future Traffic
↓
Increase Capacity
Simple ga gurthupettuko:
Predictive Scaling = **Predict future demand and prepare capacity**

---

15. What is an Auto Scaling Launch Template?
Answer:
A Launch Template contains the configuration needed to launch EC2 instances.
It can define things such as AMI, instance type, key pair, security groups, storage, and other EC2 settings.
The Auto Scaling Group uses the Launch Template when it needs to launch new instances.
For example:
Launch Template
↓
AMI
Instance Type
Security Group
Storage
↓
Auto Scaling Group
↓
EC2
Simple ga gurthupettuko:
Launch Template = **How the new EC2 should be created**

---

16. What is the difference between Launch Template and Auto Scaling Group?
Answer:
A Launch Template defines the configuration of an EC2 instance.
An Auto Scaling Group manages how many instances should run and where they should run.
So, the Launch Template tells ASG **what kind of EC2 to launch**, while ASG decides **how many EC2 instances are needed**.
Example:
Launch Template
↓
What EC2 should look like

Auto Scaling Group
↓
How many EC2 instances should run
Simple ga gurthupettuko:
Launch Template = **EC2 configuration**
ASG = **EC2 count and management**

---

17. What is an AMI in Auto Scaling?

Answer:
An AMI, or Amazon Machine Image, is a template used to create EC2 instances.
In an Auto Scaling Group, the AMI is commonly specified through the Launch Template.
It contains the operating system and required software configuration needed when launching the instance.
For example, we can create an AMI with our application software installed and use that AMI to launch new instances.
Example:
AMI
↓
Launch Template
↓
Auto Scaling Group
↓
New EC2
↓
Application Ready
Simple ga gurthupettuko:
AMI = **Template/image used to create EC2**

---

18. How does Auto Scaling know when an EC2 instance is unhealthy?
Answer:
The Auto Scaling Group can use EC2 health checks and, when configured with a Load Balancer, Elastic Load Balancing health checks.
If an instance is considered unhealthy, the ASG can terminate it and launch a replacement to maintain the required capacity.
For example:
EC2-1
↓
Unhealthy ❌
↓
ASG
↓
Terminate
↓
Launch New EC2
Simple ga gurthupettuko:
Unhealthy EC2 = **ASG replaces it**

---

19. What is a Health Check in Auto Scaling?
Answer:
A health check helps the Auto Scaling Group determine whether an EC2 instance is healthy and should continue running as part of the group.
The ASG can use EC2 health checks and can also use Elastic Load Balancing health checks when configured.
For example, if the Load Balancer reports that an instance is not serving requests correctly, the ASG can replace that instance depending on the configuration.
Example:
Load Balancer
↓
Health Check
↓
EC2
↓
Unhealthy ❌
↓
ASG replaces EC2
Simple ga gurthupettuko:
Health Check = **Find unhealthy instances**

---

20. What is an Auto Scaling Group Availability Zone?
Answer:
An Auto Scaling Group can launch EC2 instances across multiple Availability Zones.
This helps improve application availability because all instances do not depend on only one AZ.
For example, we can configure an ASG to use subnets in AZ-1 and AZ-2.
Example:
Auto Scaling Group
|

| |
AZ-1 AZ-2
| |
EC2 EC2
Simple ga gurthupettuko:
Multiple AZs = **Better availability**

21. What is the difference between Auto Scaling and Load Balancer?
Answer:
An Auto Scaling Group manages the number and health of EC2 instances.
A Load Balancer distributes incoming application traffic across available healthy instances.
They are commonly used together.
For example:
Users
↓
Load Balancer
↓

↓ ↓ ↓
EC2-1 EC2-2 EC2-3
↑ ↑ ↑
Auto Scaling
If traffic increases, ASG can launch more instances, and the Load Balancer can send traffic to the available healthy instances.
Simple ga gurthupettuko:
Load Balancer = **Distribute traffic**
ASG = **Manage EC2 count**

---

22. What happens when traffic suddenly increases?
Answer:
When application traffic increases, a configured scaling policy can detect the increase through a metric.
The Auto Scaling Group can then launch additional EC2 instances, up to the configured maximum capacity.
For example:
Traffic Increases
↓
Metric Increases
↓
Scaling Policy
↓
ASG
↓
Launch More EC2
Simple ga gurthupettuko:
High Traffic = **Scale Out**

---

23. What happens when traffic decreases?
Answer:
When traffic decreases, a configured scaling policy can reduce the number of EC2 instances.
The ASG can terminate extra instances while respecting the configured minimum capacity.
For example:
Traffic Decreases
↓
Metric Decreases
↓
Scaling Policy
↓
ASG
↓
Terminate Extra EC2
Simple ga gurthupettuko:
Low Traffic = **Scale In**

---

24. What is Instance Refresh?
Answer:
Instance Refresh is used to replace instances in an Auto Scaling Group with new instances based on an updated configuration.
It is useful when we update things such as the AMI or Launch Template and want the instances in the group to use the new configuration.
For example, suppose we have an old application version on our EC2 instances. We create a new AMI and update the Launch Template. Instance Refresh can gradually replace old instances with new ones.
Example:
Old EC2
↓
New AMI / Launch Template
↓
Instance Refresh
↓
New EC2 instances
Simple ga gurthupettuko:

Instance Refresh = **Replace old instances with updated instances**

---

25. What is a Warm Pool in Auto Scaling?
Answer:
A Warm Pool keeps pre-initialized EC2 instances ready for future use.
It can reduce the time needed to provide capacity when the Auto Scaling Group needs to scale out.
This is useful when EC2 instances take a long time to start and become ready.
For example:
Auto Scaling Group
↓
Warm Pool
↓
Ready / Partially Ready EC2
↓
Use when scaling is needed
Simple ga gurthupettuko:
Warm Pool = **Keep instances ready for faster scaling**

---

26. What is Lifecycle Hook?
Answer:
A Lifecycle Hook allows us to pause an EC2 instance during certain Auto Scaling lifecycle events.
This gives us time to perform custom actions before the instance enters or leaves service.
Common lifecycle events include launching and terminating instances.
For example, when an instance is launching, we can pause it and perform some setup before allowing it to continue.
Example:
ASG
↓
Launch EC2
↓
Lifecycle Hook
↓
Custom Setup
↓
EC2 In Service
Simple ga gurthupettuko:
Lifecycle Hook = **Pause EC2 lifecycle for custom work**

---

27. What is the difference between Scaling Policy and Lifecycle Hook?
Answer:
A Scaling Policy decides when to increase or decrease the number of EC2 instances.
A Lifecycle Hook allows us to pause an instance during launch or termination so that custom actions can be performed.
For example:
Scaling Policy
↓
Add EC2

Lifecycle Hook
↓
Pause EC2
↓
Run custom setup
Simple ga gurthupettuko:
Scaling Policy = **How many instances**
Lifecycle Hook = **Pause during instance lifecycle**

---

28. What is Capacity Rebalancing?
Answer:
Capacity Rebalancing helps an Auto Scaling Group maintain capacity when using Spot Instances.
If AWS indicates that a Spot Instance may be interrupted, the Auto Scaling Group can launch a replacement instance to help maintain capacity.
For example:
Spot EC2
↓
Interruption Warning
↓
ASG
↓
Launch Replacement
↓
Maintain Capacity
Simple ga gurthupettuko:

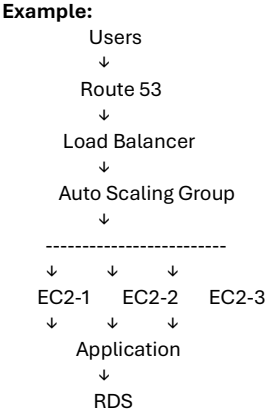
Capacity Rebalancing = **Help maintain capacity when Spot Instances are interrupted**

---

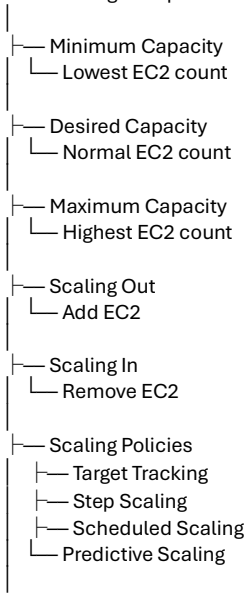
29. What are Tags in Auto Scaling Group?
Answer:
Tags are key-value pairs used to identify and organize resources.
Tags configured for an Auto Scaling Group can be propagated to instances launched by the group when configured to do so.
For example:
Environment = Production
Application = Payment
Team = Backend
These tags help us identify which application or environment an EC2 instance belongs to.
Example:
ASG
↓
EC2
↓
Environment = Production
Application = Payment
Simple ga gurthupettuko:
Tags = **Identify and organize resources**

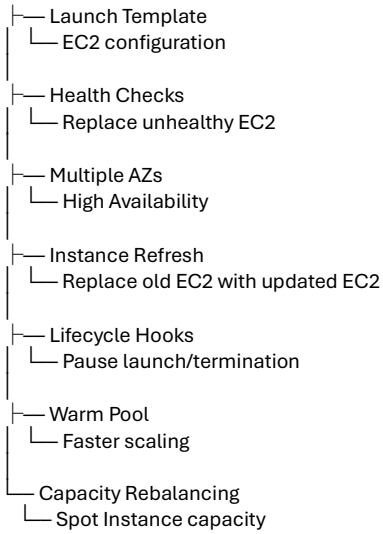
---

30. Explain a complete Auto Scaling architecture.
Answer:
A common production setup uses Route 53, a Load Balancer, an Auto Scaling Group, multiple EC2 instances, and sometimes RDS. Route 53 sends users to the Load Balancer. The Load Balancer sends traffic to healthy EC2 instances. The Auto Scaling Group maintains the required number of EC2 instances and can scale them based on traffic or other metrics.
For example, if traffic increases, the ASG launches more EC2 instances. The Load Balancer then starts sending traffic to the new healthy instances.
Example:



Simple ga gurthupettuko:
ASG = **Maintain EC2 + Auto Scaling + Replace unhealthy instances**
🔥 **Super-short Auto Scaling memory**





Final line to remember:
Auto Scaling Group = Automatically maintain, add, remove, and replace EC2 instances based on application needs.